

PaperPilot: An Evidence-Grounded Assistant for Efficient Academic Paper Reading

Abstract

Researchers and students face an increasing volume of academic literature, while traditional linear reading is time-consuming and makes it difficult to locate evidence for a specific question. This paper presents PaperPilot, an evidence-grounded paper reading assistant that combines document parsing, structural analysis, extractive summarization, keyword scoring, and paragraph retrieval. The system produces a structured overview and answers questions with paragraph-level evidence identifiers. A lightweight offline pipeline makes the prototype reproducible in classroom demonstrations without a paid model API.

1 Introduction

Academic reading is not merely a summarization problem. Readers need to identify the research question, understand the method, inspect experimental results, and return to the exact passage that supports an interpretation. Existing general chat systems may produce fluent answers but can omit evidence or become unavailable when network access is limited. We therefore study how a compact local agent can improve the first-pass reading workflow while keeping its outputs auditable.

The main contributions are threefold. First, we propose a complete local pipeline for PDF text extraction, paper structure recognition, keyword extraction, and structured summarization. Second, we introduce evidence-grounded question answering that links each answer to the most relevant source paragraphs. Third, we build a dependency-light web interface that can be demonstrated without an external database or cloud service.

2 Related Work

Previous work on scientific document understanding includes citation analysis, neural summarization, retrieval-augmented generation, and scholarly question answering. Large language models provide strong semantic understanding, but their cost and non-deterministic output can complicate introductory laboratory courses. Extractive approaches remain useful when reproducibility and source fidelity are primary requirements.

3 Method

The system follows a five-stage pipeline. The parser first extracts text from PDF, TXT, or Markdown input and normalizes whitespace. The segmenter separates paragraphs and sentences, then recognizes common headings such as Abstract, Introduction, Method, Experiments, Results, and Conclusion. The analyzer calculates term frequency with stop-word filtering and scores sentences using keyword overlap, position, length, and scientific signal words. The question-answering module ranks paragraphs by lexical overlap with the user question and returns the top evidence passages. Finally, the exporter transforms the analysis into a reusable Markdown reading note.

To reduce unsupported answers, the agent does not invent a response when semantic evidence is unavailable. It instead returns the highest-ranking source paragraphs and exposes their paragraph identifiers. This design encourages users to verify important claims in the original text.

4 Experiments

We created a small evaluation set containing 30 English and Chinese research-style documents from computer science course materials. Each document was paired with five reading questions, producing 150 question-answer cases. Three students manually judged whether the returned evidence contained the information needed to answer each question. We compared the complete pipeline with a keyword-only baseline that returned the first matching paragraph.

5 Results

The complete pipeline achieved an evidence hit rate of 86.0%, compared with 68.7% for the keyword-only baseline, an absolute improvement of 17.3 percentage points. The median processing time for a 6,000-word text paper was 0.42 seconds on a laptop. In a short usability questionnaire, participants rated the structured overview 4.4 out of 5 and the evidence identifiers 4.6 out of 5. Errors mainly occurred in scanned PDFs, multi-column extraction order, and questions that required interpreting figures or equations.

6 Discussion

The experiment shows that a lightweight system can support first-pass paper reading without relying on a remote model. However, lexical retrieval cannot fully resolve paraphrases, cross-section reasoning, or mathematical derivations. A future version can add optional embedding retrieval and an OpenAI-compatible language model while preserving the local evidence layer.

7 Conclusion

PaperPilot converts an uploaded paper into a structured, searchable reading workspace. Its key design principle is that speed should not remove traceability: summaries remain extractive, and answers are paired with source paragraph identifiers. The prototype is suitable for classroom demonstrations and provides a clear foundation for future semantic retrieval and multimodal paper understanding.